H2D、D2H 与 CPU – GPU 协同：计算、内存、通信和流水线优化

> **原始问题：**详细讲解D2H,H2D，以及GPU和CPU各自的运算和通信特点，还有两者之间的互相通信，什么时候应用，可以得到什么方面的优化提升

提问：2026/08/03 11:42　更新：2026/08/03 11:50　重要度：5/5

H2D、D2H 与 CPU – GPU 协同：计算、内存、通信和流水线优化

一句话结论

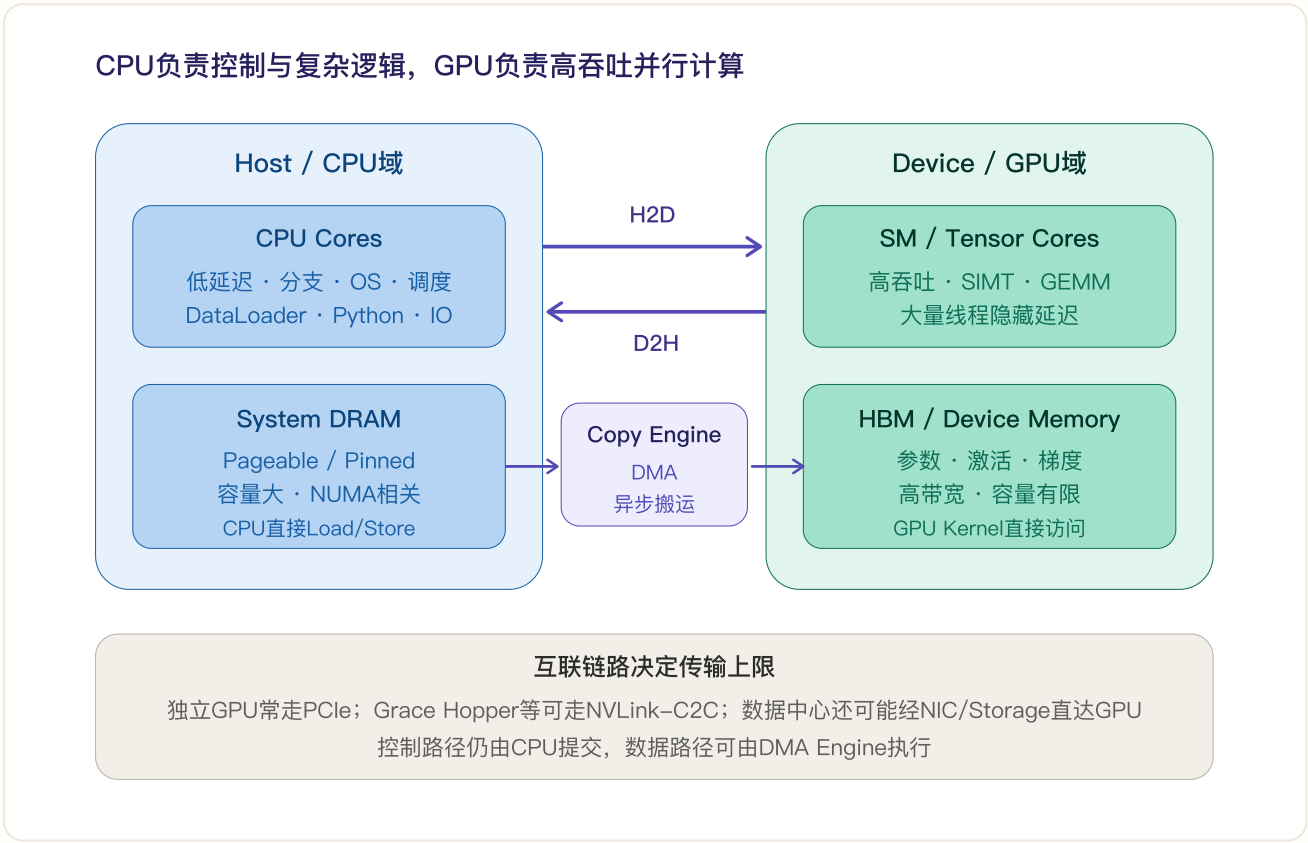
- **H2D（Host-to-Device）**：把数据从 CPU 侧系统内存搬到 GPU 侧设备内存；典型场景是把训练 Batch、推理输入、模型参数或临时 Buffer 送到 GPU。
- **D2H（Device-to-Host）**：把结果从 GPU 设备内存搬回 CPU 系统内存；典型场景是返回推理结果、保存 Checkpoint、CPU 后处理、日志和调试。
- 对独立 GPU，H2D/D2H 通常经过 PCIe 或类似 CPU–GPU 互联，由 GPU Copy Engine/DMA 搬运。**CPU 提交控制命令，不等于 CPU Core 逐字节执行整个传输。**
- CPU 擅长低延迟、复杂控制流、分支、OS/IO 和少量串行工作；GPU 擅长规则、高并行、高算术强度的大批量计算。
- 最有效的优化顺序通常是：

```
尽量不传
→ 数据留在GPU
→ 合并小传输
→ 使用提前准备好的Pinned Memory
→ non_blocking异步提交
→ 独立Stream + Event
→ 双缓冲/预取，将H2D与当前Batch计算重叠
→ 做NUMA和拓扑绑定
→ 特定场景考虑Unified Memory或GPUDirect
```

最重要的判断式：

```
把一个操作放到GPU是否值得
≈ GPU计算节省的时间
  是否大于
  H2D + D2H + Launch + Synchronization额外时间
```

一、系统位置：CPU 和 GPU 各自在哪里工作



1.1 Host 与 Device

CUDA 语境中：

```
Host   通常指CPU及其系统内存
Device 通常指GPU及其设备内存
```

独立 GPU 系统中，两套内存物理上分离：

```
CPU Core ↔ CPU Cache ↔ System DRAM
                       ↕ PCIe / NVLink-C2C等
GPU SM   ↔ L1/L2     ↔ GPU HBM
```

CPU 通常负责：

- Python/C++ 控制流；
- DataLoader、Tokenizer、解码和数据增强；
- 文件系统、网络和磁盘 IO；
- Kernel Launch；
- CUDA Stream/Event 编排；
- 分布式进程管理；
- GPU 不能高效执行的少量、分支密集任务。

GPU 通常负责：

- GEMM、Attention、卷积；
- 大规模逐元素运算；
- Reduction；
- Embedding、MoE Expert 等高并行算子；
- 前向、反向和优化器 Kernel；
- NCCL 通信 Kernel。

二、CPU 与 GPU 的计算特点

2.1 CPU：少量强核，优化单线程和复杂控制

CPU 的设计重点通常是：

- 高单线程性能；
- 大 Cache；
- 复杂分支预测；
- Out-of-Order Execution；
- 低延迟访问系统资源；
- OS、文件、网络、线程和中断处理；
- SIMD 向量指令处理适量数据并行。

适合 CPU 的任务：

- Tokenizer、JSON/文本解析；
- 动态数据结构；
- 分支密集算法；
- 非常小的 Tensor；
- 单次低延迟任务；
- 数据读取、解压、增强；
- 调度、RPC、Serving 控制面；
- 需要访问大量主机内存但计算复用很低的任务。

2.2 GPU：大量并行执行单元，优化吞吐

GPU 的设计重点通常是：

- 大量 SM 和 Warp；
- SIMT 执行；
- 高 HBM 带宽；
- Tensor Core；
- 用大量 Ready Warp 隐藏访存延迟；
- 用大批量规则计算摊薄 Kernel Launch 和调度开销。

适合 GPU 的任务：

- 大矩阵乘；
- Attention；
- 大 Batch 的向量化计算；
- 相同操作应用到大量元素；
- 高算术强度；
- 能够在 HBM、Shared Memory、Register 中复用数据的计算。

2.3 CPU 与 GPU 对比

维度	CPU	GPU
优化目标	低延迟、通用性	高吞吐、并行性

维度	CPU	GPU
核心数量	较少但强	大量轻量执行单元
控制流	擅长复杂分支	Warp分歧会损失效率
Cache	单核/共享Cache较强	更依赖HBM带宽和显式复用
内存容量	系统DRAM通常更大	HBM容量较小但带宽高
启动成本	函数执行直接	Kernel Launch有固定开销
最佳工作粒度	小任务、串行/控制	大任务、大Batch、规则计算
IO/OS	可直接处理	通常由CPU/驱动提交
典型LLM角色	数据、控制、Serving调度	模型计算和GPU通信

结论不是“GPU 永远比 CPU 快”，而是：

```
任务必须有足够计算量、并行度和数据复用
才能摊薄CPU↔GPU传输与Kernel Launch成本
```

三、H2D 和 D2H 的精确定义

3.1 H2D

```
Host Memory → Device Memory
```

CUDA C++：

```
cudaMemcpy(
    device_ptr,
    host_ptr,
    bytes,
    cudaMemcpyHostToDevice
);
```

PyTorch：

```
gpu_tensor = cpu_tensor.to("cuda")
```

典型用途：

- 训练 Batch；

- 推理输入；
- 初始化模型参数；
- CPU 预处理后的特征；
- CPU Offload 状态重新加载；
- Checkpoint 恢复。

3.2 D2H

```
Device Memory → Host Memory
```

CUDA C++：

```
cudaMemcpy(
    host_ptr,
    device_ptr,
    bytes,
    cudaMemcpyDeviceToHost
);
```

PyTorch：

```
cpu_tensor = gpu_tensor.to("cpu")
```

典型用途：

- 推理结果返回业务代码；
- GPU 结果交给 CPU 后处理；
- 保存模型或优化器状态；
- 日志、Metric、可视化；
- 调试；
- CPU Offload；
- GPU 生成数据写入普通 CPU IO 栈。

3.3 方向名是相对谁而言

```
H2D = CPU内存到GPU内存
D2H = GPU内存到CPU内存
```

```
D2D = GPU内存到GPU内存
H2H = CPU内存到CPU内存
```

在多 GPU 系统中，GPU↔GPU 还可能走：

- NVLink/NVSwitch；
- PCIe P2P；
- GPUDirect RDMA + NIC；
- 无 P2P 时退化为 GPU→Host→GPU Bounce Path。

四、传输到底由谁执行：DMA 与 Copy Engine

4.1 CPU 是控制面，DMA 是数据面

常见异步 H2D：

```
CPU调用cudaMemcpyAsync
→ CUDA Runtime/Driver建立传输描述
→ 命令进入CUDA Stream
→ GPU Copy Engine/DMA读取Pinned Host Memory
→ 数据经过PCIe/NVLink-C2C
→ 写入GPU HBM
→ Event标记完成
```

CPU Core 不需要在整个传输期间逐字节循环复制，因此提交后可以继续：

- 处理下一 Batch；
- 发起其他 CUDA 操作；
- 执行业务逻辑；
- 准备其他 GPU 的输入。

但对 Pageable Memory，驱动通常需要额外的 CPU 内存复制到 Pinned Staging Buffer，CPU/DRAM 仍会参与。

4.2 Copy Engine 与 Compute Engine

GPU 可能拥有独立 Copy Engine：

- H2D Copy Engine；
- D2H Copy Engine；
- SM/Tensor Core 负责计算。

某些设备允许：

```
一个H2D
+ 一个D2H
+ GPU Kernel
```

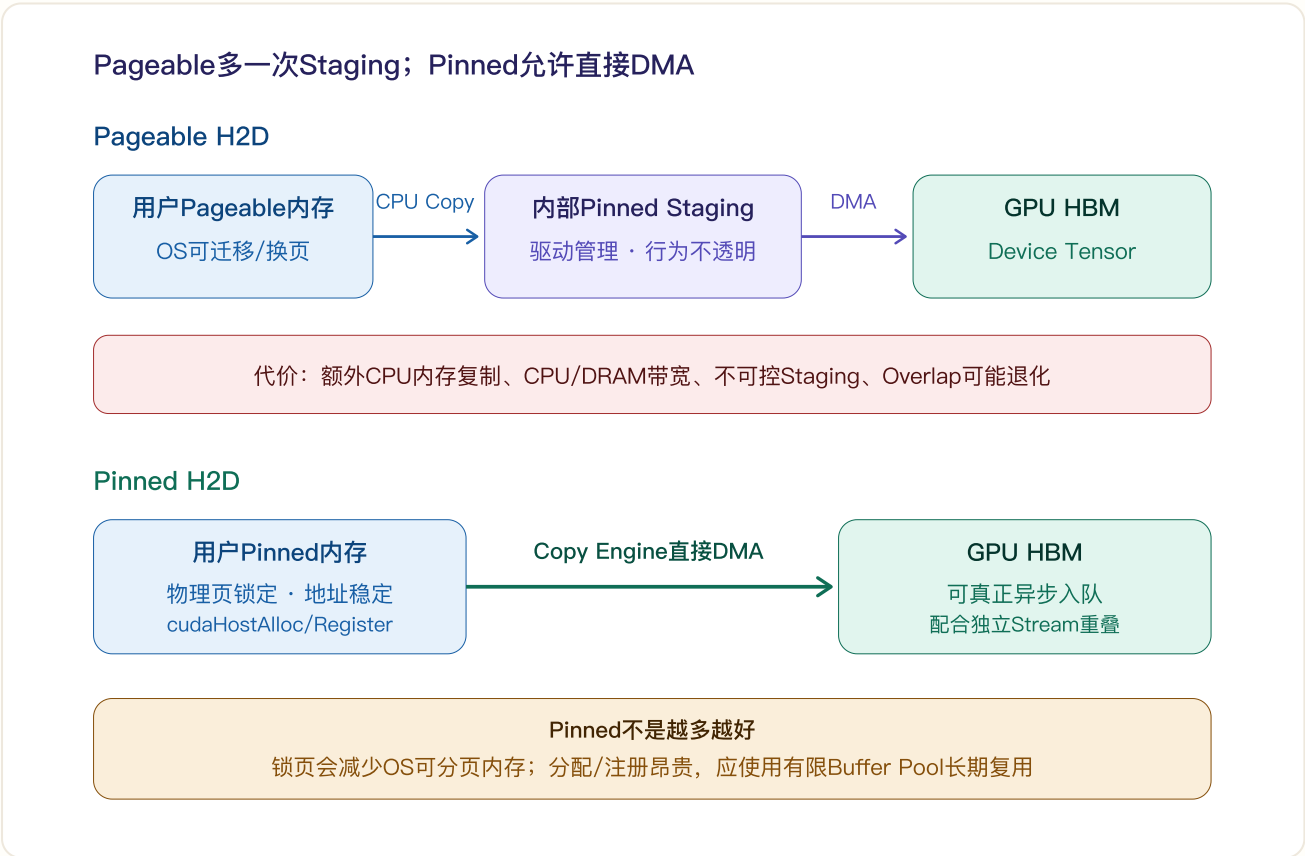
同时推进。

但能否并发必须查询并实测：

```
cudaDeviceProp prop;
cudaGetDeviceProperties(&prop, device);
printf("asyncEngineCount=%d\n", prop.asyncEngineCount);
```

`asyncEngineCount` 表示硬件能力，不保证应用自动获得重叠。

五、Pageable Memory 与 Pinned Memory



5.1 Pageable Memory

普通 `malloc` 或普通 CPU Tensor 通常位于 Pageable Memory。

OS 可以：

- 迁移物理页；
- 换页；
- 改变虚拟页到物理页的映射。

GPU DMA 需要传输期间物理页稳定，因此 Pageable H2D 常见路径为：

```
Pageable User Buffer
→ CPU memcpy到内部Pinned Staging
→ DMA到GPU HBM
```

代价：

- 多一次 CPU Memory Copy；
- 消耗 System DRAM 带宽；
- 增加 CPU 开销；
- 延迟更不稳定；
- 真正异步和 Copy–Compute Overlap 可能退化。

具体 Staging Buffer 的大小、复用策略和同步时机属于驱动实现细节，应用不应依赖。

5.2 Pinned / Page-Locked Memory

Pinned Memory 的物理页被锁定，传输期间不会被 OS 换出或迁移。

CUDA API：

```
cudaHostAlloc(&ptr, bytes, flags);
```

或注册已有内存：

```
cudaHostRegister(ptr, bytes, flags);
```

PyTorch：

```
x = torch.empty(
    shape,
    pin_memory=True,
)
```

或者：

```
loader = DataLoader(
    dataset,
    pin_memory=True,
)
```

优势：

- Copy Engine 可直接将其作为 DMA 源/目标；
- H2D/D2H 带宽更高且更可预测；
- 是可靠 Copy–Compute Overlap 的关键条件；
- 避免驱动内部 Pageable→Pinned Staging。

5.3 Pinned Memory 的代价

Pinned Memory 不是越多越好：

- 不能被 OS 正常换页；
- 占用宝贵物理 RAM；
- 大量锁页会影响整个系统；
- `cudaHostAlloc`、`cudaHostRegister` 和取消注册相对昂贵；
- 频繁 Pin/Unpin 可能比直接 Pageable 复制更慢。

最佳实践：

```
建立有限大小Pinned Buffer Pool
→ 长期复用
→ 不在每个Batch热路径里临时Pin/Unpin
```

5.4 为什么 `x.pin_memory().to("cuda")` 可能更慢

它包含两步：

```
Pageable x
→ CPU复制出一个新的Pinned x_pinned
```

```
→ H2D复制到GPU
```

总时间：

```
T_total
= T_pin_copy + T_H2D_pinned
```

只有满足：

```
T_pin_copy + T_H2D_pinned
< T_H2D_pageable
```

才有单次收益。

通常更合理的是：

```
DataLoader后台线程提前Pin下一Batch
+ 主线程non_blocking传输当前Batch
```

这样 Pinning 开销可以被 CPU 数据准备和 GPU 计算隐藏。

六、同步复制、异步提交和真正重叠是三件不同的事

6.1 同步复制

```
gpu = cpu.to("cuda")
```

从调用者视角，PyTorch 通常会在合适位置等待，使调用安全地完成后再继续。

优点：

- 简单；
- 正确性直观。

缺点：

- CPU 主线程等待；
- 难以形成流水线；
- 每个小 Tensor 都同步时，固定开销很高。

6.2 非阻塞提交

```
gpu = cpu.to(
    "cuda",
    non_blocking=True,
)
```

这首先意味着：

> CPU 不在每次复制后立刻等待 CUDA Stream 完成。

它可以减少：

- Host 线程阻塞；
- 每 Tensor 的 `cudaStreamSynchronize`；
- 多个小复制之间的提交间隙。

但 `non_blocking=True` 不自动保证：

- H2D 与 Kernel 在 GPU 上同时执行；
- Pageable Memory 不需要 Staging；
- 跨 Stream 数据依赖自动正确；
- D2H 结果可以被 CPU 立即读取。

6.3 真正的 Copy – Compute Overlap

通常需要同时满足：

1. Host Buffer 是 Pinned Memory；
2. 使用 `cudaMemcpyAsync` / `non_blocking=True`；
3. Copy 与独立计算位于不同非默认 Streams；
4. GPU 有可用 Copy Engine；
5. Copy 与 Compute 没有直接数据依赖；
6. 使用 Event/Stream Wait 表达必要依赖；
7. 数据块足够大，能摊薄提交开销；
8. PCIe、DRAM、HBM 和 SM 没有形成更严重的共享瓶颈。

七、统一数值算例：256 MiB Batch

统一输入：

```
B = 8
S = 4096
H = 4096
Dtype = BF16 = 2 Byte

Shape = [B,S,H]
```

元素数：

```
N
= 8 × 4096 × 4096
= 134,217,728 elements
```

数据量：

```
Bytes
= 134,217,728 × 2
= 268,435,456 Byte
= 256 MiB
≈ 0.268 GB
```

假设实测有效 H2D 带宽：

```
BW_H2D = 24 GB/s
```

则：

```
T_H2D
= 0.268 GB ÷ 24 GB/s
≈ 0.01118 s
≈ 11.18 ms
```

变量与单位：

```
Bytes     传输字节数        单位：GB
BW_H2D    H2D有效带宽       单位：GB/s
T_H2D     传输时间          单位：s

量纲：GB ÷ (GB/s) = s
```

假设该 Batch 的 GPU Compute 为：

```
T_compute = 8 ms
```

7.1 串行

```
T_serial
= T_H2D + T_compute
= 11.18 + 8
= 19.18 ms/batch
```

稳态吞吐：

```
Throughput_serial
≈ 1000 ÷ 19.18
≈ 52.1 batch/s
```

7.2 完美双缓冲

```
T_pipeline_steady
≈ max(T_H2D, T_compute)
= max(11.18, 8)
= 11.18 ms/batch
```

稳态吞吐：

```
Throughput_overlap
≈ 1000 ÷ 11.18
≈ 89.4 batch/s
```

理想吞吐提升：

```
Speedup
≈ 19.18 ÷ 11.18
≈ 1.72×
```

这里 GPU 仍有约：

```
11.18 − 8 = 3.18 ms
```

等待下一 Batch，因为 H2D 比 Compute 慢。

如果 Compute 改为 15 ms：

```
T_steady ≈ max(11.18,15) = 15 ms
```

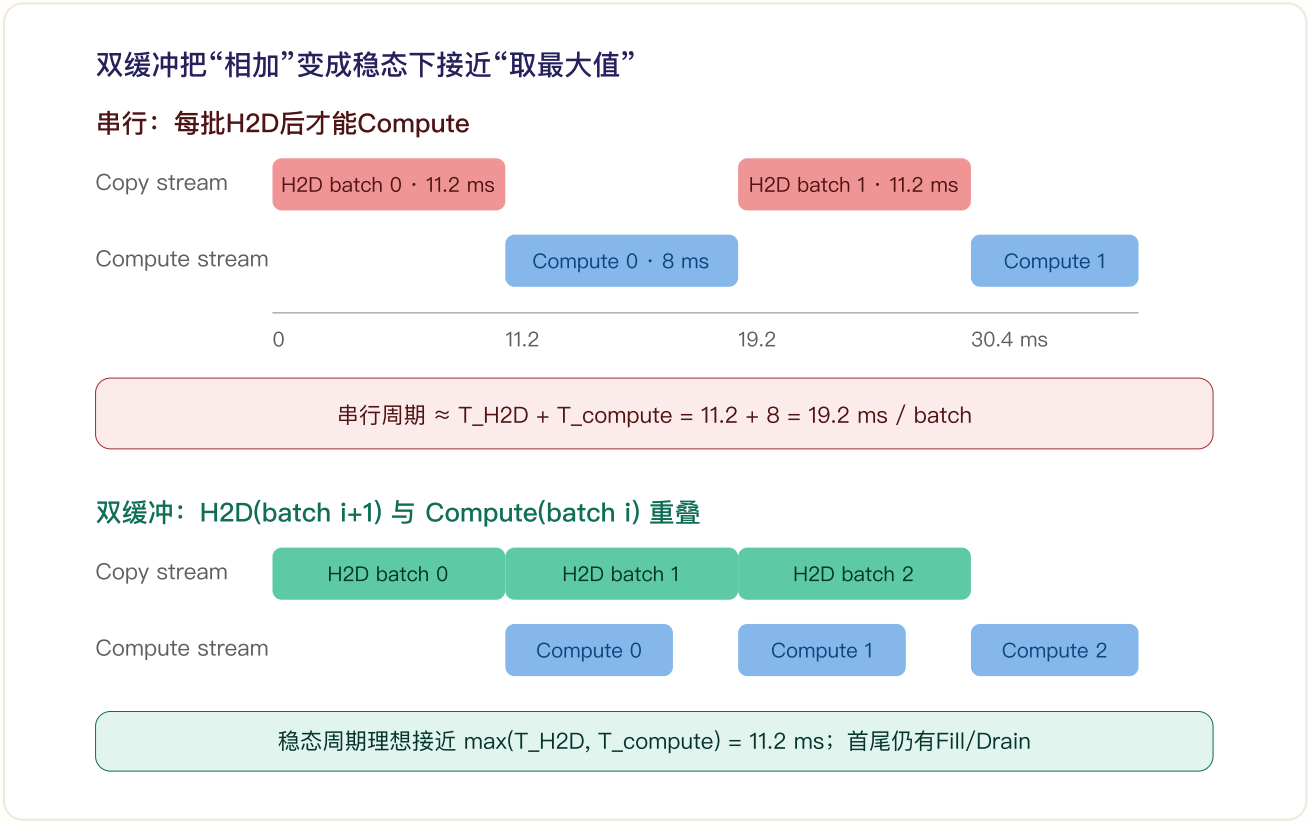
此时 H2D 可被计算完全隐藏。

7.3 为什么不是每一批都只要 11.18 ms

完整流水线还有：

- 第一个 Batch 的 Fill；
- 最后一个 Batch 的 Drain；
- Event/Launch 开销；
- DataLoader 抖动；
- Buffer 轮换；
- PCIe 与其他设备竞争；
- Pageable→Pinned、预处理和数据解码。

因此 `max(T_copy,T_compute)` 是长流水线的理想稳态下界，不是所有 Batch 的精确延迟。



八、H2D 和 D2H 的正确同步规则

8.1 Pinned H2D 后，CPU 不能立即修改源 Buffer

```
cpu = torch.ones(
    1024,
    pin_memory=True,
)

gpu = cpu.to(
    "cuda",
    non_blocking=True,
)

# 危险：DMA可能还在读取cpu
cpu.zero_()
```

正确做法是等对应复制 Event/Stream 完成后再复用 Pinned Buffer。

8.2 同一个 CUDA Stream 中，后续 GPU 操作自动等待

```
gpu = cpu.to(
    "cuda",
    non_blocking=True,
)

# 若在同一Stream提交，matmul按序等待H2D完成
output = model(gpu)
```

不需要在两者之间立刻全局同步。

8.3 跨 Stream 使用需要显式依赖

```
copy_stream = torch.cuda.Stream()
compute_stream = torch.cuda.current_stream()

with torch.cuda.stream(copy_stream):
    gpu = cpu.to(
        "cuda",
        non_blocking=True,
    )
    copy_done = copy_stream.record_event()

compute_stream.wait_event(copy_done)
output = model(gpu)
```

`wait_event()` 建立 GPU 侧顺序，不阻塞 CPU。

8.4 D2H 后 CPU 读取前必须确认完成

```
cpu_result = gpu_result.to(
    "cpu",
    non_blocking=True,
)

# 不能假定这里已经完成D2H
# print(cpu_result) 可能过早读取

torch.cuda.synchronize()
print(cpu_result)
```

更细粒度做法是同步负责 D2H 的 Event，而不是全设备同步。

8.5 `wait_stream()` 和 `record_stream()` 解决不同问题

API	解决的问题
`wait_stream()` / `wait_event()`	数据和执行依赖：消费者何时可以运行
`record_stream()`	内存生命周期：Caching Allocator何时可以复用底层Storage

如果 Tensor 在非创建 Stream 上使用，应考虑：

```
tensor.record_stream(other_stream)
```

否则原 Python 引用消失后，Caching Allocator 可能过早复用仍被异步操作使用的内存。

九、PyTorch DataLoader 的标准 H2D 路径

```
import torch
from torch.utils.data import DataLoader

loader = DataLoader(
    dataset,
    batch_size=8,
    num_workers=8,
    pin_memory=True,
    persistent_workers=True,
    prefetch_factor=2,
)

model = model.cuda()
```

```
for batch in loader:
    inputs, labels = batch

    inputs = inputs.to(
        "cuda",
        non_blocking=True,
    )

    labels = labels.to(
        "cuda",
        non_blocking=True,
    )

    output = model(inputs)
    loss = loss_fn(output, labels)
```

各参数作用：

```
num_workers
→ 多进程并行读取、解码、增强

pin_memory=True
→ 后台Pin Memory线程将Batch放入Pinned Host Memory

persistent_workers=True
→ Epoch之间不反复创建Worker

prefetch_factor
→ Worker预先准备多个Batch

non_blocking=True
→ 主线程提交H2D后不立即等待
```

注意：这能改善 CPU Pipeline，但不保证 H2D 一定与当前 GPU Compute 重叠。要确保重叠，通常还需要显式预取到 Side Stream。

十、PyTorch 双缓冲预取器

下面展示核心机制；生产代码要扩展到 Nested Dict/List、多 GPU、异常处理和 CUDA Graph。

```
import torch


def move_to_cuda(batch, device, stream):
    with torch.cuda.stream(stream):
        if isinstance(batch, torch.Tensor):
            return batch.to(
                device,
                non_blocking=True,
            )

        if isinstance(batch, (list, tuple)):
            return type(batch)(
                move_to_cuda(x, device, stream)
                for x in batch
            )

        if isinstance(batch, dict):
```

```
            return {
                k: move_to_cuda(v, device, stream)
                for k, v in batch.items()
            }

        return batch


def record_on_stream(batch, stream):
    if isinstance(batch, torch.Tensor):
        batch.record_stream(stream)

    elif isinstance(batch, (list, tuple)):
        for x in batch:
            record_on_stream(x, stream)

    elif isinstance(batch, dict):
        for x in batch.values():
            record_on_stream(x, stream)


class CUDAPrefetcher:
    def __init__(self, loader, device="cuda"):
        self.loader = iter(loader)
        self.device = torch.device(device)
        self.copy_stream = torch.cuda.Stream(
            device=self.device,
        )

        self.next_batch = None
        self._preload()

    def _preload(self):
        try:
            cpu_batch = next(self.loader)
        except StopIteration:
            self.next_batch = None
            return

        self.next_batch = move_to_cuda(
            cpu_batch,
            self.device,
            self.copy_stream,
        )

    def next(self):
        current = torch.cuda.current_stream(
            self.device,
        )

        # 当前Compute Stream等待下一Batch H2D完成
        current.wait_stream(self.copy_stream)

        batch = self.next_batch

        if batch is None:
            return None

        # 告诉Caching Allocator：当前Stream仍在使用这些Tensor
        record_on_stream(batch, current)

        # 立即在Side Stream预取再下一批
        self._preload()

        return batch


prefetcher = CUDAPrefetcher(loader)

while True:
    batch = prefetcher.next()

    if batch is None:
```

```
        break

    inputs, labels = batch

    # 计算当前Batch时，下一Batch正在copy_stream做H2D
    output = model(inputs)
    loss = loss_fn(output, labels)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
```

时间关系：

```
H2D(batch 0)
→ Compute(batch 0) || H2D(batch 1)
→ Compute(batch 1) || H2D(batch 2)
→ ...
```

十一、CUDA C++ 双缓冲示例

```
#include <cuda_runtime.h>

constexpr int NUM_BUFFERS = 2;

float* h_buf[NUM_BUFFERS];
float* d_buf[NUM_BUFFERS];
cudaStream_t streams[NUM_BUFFERS];

for (int i = 0; i < NUM_BUFFERS; ++i) {
    cudaHostAlloc(
        &h_buf[i],
        bytes,
        cudaHostAllocDefault
    );

    cudaMalloc(
        &d_buf[i],
        bytes
    );

    cudaStreamCreate(
        &streams[i]
    );
}

for (int batch = 0; batch < num_batches; ++batch) {
    int slot = batch % NUM_BUFFERS;

    // 复用slot前，确保它上一轮的copy和kernel完成
    cudaStreamSynchronize(streams[slot]);

    fill_cpu_batch(
        h_buf[slot],
        batch
    );

    cudaMemcpyAsync(
        d_buf[slot],
        h_buf[slot],
        bytes,
        cudaMemcpyHostToDevice,
```

```
        streams[slot]
    );

    kernel<<<grid, block, 0, streams[slot]>>>(
        d_buf[slot]
    );
}

for (int i = 0; i < NUM_BUFFERS; ++i) {
    cudaStreamSynchronize(streams[i]);
    cudaStreamDestroy(streams[i]);
    cudaFree(d_buf[i]);
    cudaFreeHost(h_buf[i]);
}
```

同一个 Stream 内：

```
H2D(slot i)
→ Kernel(slot i)
```

严格有序。

不同 Stream：

```
Stream 0 Kernel(batch 0)
可与
Stream 1 H2D(batch 1)
重叠
```

真正的生产实现通常使用 Event 而不是在 CPU 填充前粗粒度 `cudaStreamSynchronize()`，以缩小等待范围。

十二、D2H 的高效模式

12.1 不要每 Step 把大量 Tensor 拉回 CPU

错误模式：

```
for step in range(num_steps):
    huge = activation.cpu()
    save_or_log(huge)
```

问题：

- 强制 D2H；
- 可能同步整个当前 Stream；

- 阻塞训练主循环；
- 占用 PCIe 和 CPU DRAM；
- 破坏 GPU Pipeline。

更好的方式：

- 只复制必要的标量或聚合结果；
- 在 GPU 上先 Reduction；
- 降低日志频率；
- 异步 D2H 到 Pinned Buffer；
- 用后台 CPU 线程处理完成的 Buffer；
- Checkpoint 使用分片和后台写盘。

12.2 异步 D2H 到 Pinned Destination

```
copy_stream = torch.cuda.Stream()
compute_stream = torch.cuda.current_stream()

# GPU结果y由compute_stream产生
copy_stream.wait_stream(compute_stream)

cpu_out = torch.empty(
    y.shape,
    dtype=y.dtype,
    pin_memory=True,
)

with torch.cuda.stream(copy_stream):
    cpu_out.copy_(
        y,
        non_blocking=True,
    )

    # y在copy_stream完成前不能被Allocator复用
    y.record_stream(copy_stream)
    done = copy_stream.record_event()

# CPU可以处理别的事
prepare_next_request()

# CPU第一次读取cpu_out之前等待
# 只同步这个Event，不全局同步整个GPU
done.synchronize()
consume_on_cpu(cpu_out)
```

十三、何时应把数据留在 GPU，何时送回 CPU

13.1 尽量留在 GPU

适合：

- 训练 Forward→Loss→Backward→Optimizer；
- 多个连续 GPU 算子；
- GPU 上可以完成 Reduction；
- 推理后续步骤仍在 GPU；
- GPU 上能直接执行 Sampling；
- 后续需要 NCCL 通信。

原则：

```
中间Tensor不要因为“CPU代码好写”就频繁D2H/H2D往返
```

13.2 适合 H2D

- DataLoader Batch；
- CPU Tokenizer 输出；
- CPU 数据增强结果；
- Checkpoint Restore；
- CPU Offload 参数或优化器状态回载；
- CPU/NVMe 数据到 GPU 的传统路径。

13.3 适合 D2H

- 最终业务结果需要 CPU；
- 保存 Checkpoint；
- CPU 编码、压缩或网络协议；
- 少量 Metric；

- GPU 不适合的分支密集后处理；
- Offload；
- 调试。

13.4 不适合 GPU 的“小计算”

设一个 CPU 操作只需：

```
T_cpu = 20 μs
```

若 Offload 到 GPU：

```
H2D = 15 μs
Kernel Launch + Compute = 10 μs
D2H = 15 μs
总计 = 40 μs
```

即使 GPU Kernel 本身更快，总流程反而更慢。

所以 GPU Offload 的收益边界：

```
T_cpu
>
T_H2D + T_kernel + T_D2H + T_sync
```

如果输出可继续留在 GPU，去掉 D2H 后，收益条件会更容易满足。

十四、CPU – GPU 通信优化能改善哪些指标

优化	主要改善	原理	不一定改善
减少往返传输	Latency、吞吐、CPU占用	直接删除PCIe和同步成本	GPU显存容量
合并小传输	Latency、吞吐	摊薄每次API/DMA固定开销	总Payload字节数
Pinned Memory	H2D/D2H带宽、抖动	允许直接DMA	GPU计算时间
`non_blocking=True`	CPU提交效率、吞吐	减少Host同步	单次GPU复制本身一定变快
Side Stream	吞吐	Copy与独立Compute并发	单请求关键路径延迟一定下降
双缓冲/预取	稳态吞吐、GPU利用率	下一批传输隐藏在当前计算下	首批Latency、尾部Drain
降低传输dtype	带宽、Latency	Payload减少	数值质量、固定延迟

优化	主要改善	原理	不一定改善
NUMA绑定	带宽、尾延迟	避免跨Socket DRAM流量	单Socket系统
Unified Memory Prefetch	易用性、迁移可预测性	提前迁移页面	所有访问模式
GPUDirect RDMA/GDS	CPU占用、Latency、带宽	NIC/Storage直达GPU	不支持的拓扑/文件系统

十五、NUMA：多路 CPU 服务器的隐藏瓶颈

双 Socket 服务器中：

```
GPU 0靠近NUMA Node 0
但DataLoader Worker和Pinned Buffer在NUMA Node 1
```

传输路径可能变成：

```
Node 1 DRAM
→ CPU互联UPI/Infinity Fabric
→ Node 0 PCIe Root Complex
→ GPU 0
```

结果：

- H2D 带宽下降；
- CPU 互联被占用；
- 尾延迟和抖动增加；
- 多 GPU 同时加载时拥塞。

建议：

1. 用 `nvidia-smi topo -m` 查看 GPU/CPU/NIC 拓扑；
2. 将负责 GPU 的 CPU Worker 绑到相近 NUMA Node；
3. 在绑定后分配并 First-Touch Host Buffer；
4. 再注册为 Pinned Memory；
5. 建立 GPU–CPU Worker–Pinned Buffer 局部性。

示意：

```
numactl \
  --cpunodebind=0 \
```

```
  --membind=0 \
  python train.py
```

不能只看 GPU Utilization；还要检查 CPU NUMA Miss、PCIe Throughput 和 DataLoader Wait。

十六、Unified Memory：统一地址不等于没有数据移动

CUDA Unified Memory：

```
cudaMallocManaged(&ptr, bytes);
```

CPU 和 GPU 可以使用同一指针，但底层页面仍可能在：

- CPU System Memory；
- GPU Memory；
- 两端之间迁移。

首次访问可能触发 Page Fault 和页面迁移：

```
GPU访问CPU驻留页面
→ Fault
→ Driver迁移/映射
→ Kernel继续
```

优点：

- 编程简单；
- 可超额使用 GPU 显存；
- 适合复杂、不规则数据结构；
- 某些 CPU–GPU 一致性平台更自然。

风险：

- Page Fault 延迟；
- Thrashing：CPU/GPU反复争用同一页面；
- 访问模式不规则时不可预测；
- 性能可能低于显式批量 H2D/D2H。

可使用 Prefetch：

```
cudaMemPrefetchAsync(
    ptr,
    bytes,
    gpu_id,
    stream
);
```

或预取回 CPU：

```
cudaMemPrefetchAsync(
    ptr,
    bytes,
    cudaCpuDeviceId,
    stream
);
```

关键心智模型：

> Unified Virtual Addressing/Managed Memory 统一的是寻址和管理体验，不会让 PCIe 物理搬运成本凭空消失。

在 Grace Hopper 等 CPU–GPU 高带宽一致性平台上，访问与迁移语义可能明显不同，必须按具体平台评估，不能套用普通离散 PCIe GPU 的结论。

十七、Mapped Pinned / Zero-Copy Memory

Pinned Host Memory 可以映射到 GPU 地址空间，GPU Kernel 直接访问 Host Memory。

这叫 Zero-Copy 的原因是：

```
不先显式复制整个Buffer到HBM
```

但每次 GPU Load/Store 仍可能穿过 PCIe，因此并不是“零成本”。

适合：

- 数据只访问一次；
- 稀疏、随机、小量读取；
- GPU 显存容量紧张；
- 集成 GPU 或高带宽一致性平台；
- 显式复制成本高于实际访问量。

不适合：

- 反复读取；
- 大规模 GEMM 输入；
- 高复用数据；
- 对带宽和延迟敏感的主训练热路径。

对于高复用 Tensor：

```
先H2D到HBM一次
→ GPU在HBM反复使用
```

通常比每次都远程访问 Host Memory 更好。

十八、GPUDirect：绕过CPU Bounce Buffer

18.1 GPUDirect RDMA

允许 NIC 或其他 PCIe Peer Device 直接读写 GPU Memory：

```
Remote GPU/NIC
→ RDMA
→ Local GPU HBM
```

避免：

```
NIC → CPU DRAM → GPU
```

收益：

- 降低 CPU 占用；
- 减少额外内存复制；
- 降低延迟；
- 提高网络到 GPU 的带宽。

常用于：

- NCCL 跨节点；

- 分布式训练；
- GPU Serving；
- 高性能网络数据流。

18.2 GPUDirect Storage

直接路径：

```
NVMe / Remote Storage
→ DMA
→ GPU Memory
```

避免 CPU Bounce Buffer。

适合：

- 大规模 Checkpoint；
- 数据集直接加载到 GPU；
- GPU 生成数据直接写存储；
- Storage 已成为训练瓶颈。

但控制路径仍由 CPU/驱动设置，且需要：

- 支持的 GPU/驱动；
- 支持的文件系统与存储；
- 正确拓扑；
- 内核模块/DMA-BUF；
- 对齐和注册条件。

如果条件不满足，系统可能回退到 CPU Bounce Path。

十九、训练中的典型 CPU – GPU 流水线

完整训练数据流可能是：

```
Storage
→ CPU Worker读取
→ 解码/Tokenizer/Data Augmentation
```

```
→ Pageable Batch
→ DataLoader Pin线程
→ Pinned Batch
→ H2D Copy Stream
→ GPU Input Buffer
→ Forward/Backward/Optimizer
→ 少量Metric D2H
→ CPU日志
```

理想并发：

```
CPU准备Batch i+2
||
H2D复制Batch i+1
||
GPU计算Batch i
```

这是三级 Pipeline：

1. CPU 数据准备；
2. CPU→GPU 搬运；
3. GPU 计算。

稳态周期接近三者最大值：

```
T_steady
≈ max(
    T_cpu_prepare,
    T_H2D,
    T_gpu_compute
  )
```

如果：

```
T_cpu_prepare = 20 ms
T_H2D = 11 ms
T_gpu_compute = 8 ms
```

即使 H2D 完美重叠，整体仍被 CPU DataLoader 的 20 ms 限制。

所以要同时看：

- DataLoader Queue Depth；
- CPU Worker Utilization；
- Pin Memory Thread；
- H2D Throughput；
- GPU Idle Gap。

二十、推理和 Serving 中的特点

20.1 输入 Token 通常很小

Token IDs：

```
[B,S] × int32/int64
```

与模型权重、KV Cache 和计算相比，输入 Token H2D 通常很小。

因此 LLM Serving 的主要 CPU–GPU 问题可能不是 Token H2D 带宽，而是：

- CPU 调度；
- Continuous Batching；
- Sampling；
- KV Cache 管理；
- 小 Kernel Launch；
- D2H Logits；
- 多请求同步。

20.2 不要把完整 Logits 无脑 D2H

假设：

```
Batch = 128
Vocab = 128K
FP16 Logits
```

数据量：

```
128 × 131072 × 2 Byte
= 32 MiB
```

每个 Decode Step 都 D2H 32 MiB，会产生巨大 PCIe 压力。

更好的方式：

```
GPU上完成Softmax/Top-k/Sampling
→ 只D2H每请求一个Token ID
```

假设每请求返回一个 int32：

```
128 × 4 Byte = 512 Byte
```

从 32 MiB 降到 512 Byte，差异约 65536 倍。

这就是"先在 GPU Reduction，再传最小结果"的典型优化。

二十一、常见失败模式

失败1：每个小Tensor单独 `.cuda()`

问题：

- 每次调用有固定开销；
- 多次 Stream 同步；
- DMA 传输过碎。

优化：

- 合并/打包 Tensor；
- 一次传大 Batch；
- 使用 Nested Batch 的统一异步搬运。

失败2：训练循环中临时 `pin_memory()`

问题：

- Pinning 本身需要 CPU Copy；
- 主线程阻塞；
- 频繁锁页/解锁。

优化：

- `DataLoader(pin_memory=True)`；

- Pinned Buffer Pool；
- 后台线程提前准备。

失败3：`non_blocking=True` 后立刻 `torch.cuda.synchronize()`

```
x = x.to("cuda", non_blocking=True)
torch.cuda.synchronize()
```

这会立刻消除异步收益。

只在：

- CPU 必须读取结果；
- 精确计时边界；
- 调试；

时同步。

失败4：Pinned H2D 后立即复用源Buffer

DMA 仍在读取，数据可能损坏。

失败5：D2H 后CPU立即 `.numpy()` 或打印

异步 D2H 尚未完成，必须先等待对应 Event。

失败6：Copy与Compute放在同一Stream却期待重叠

同一 Stream 严格按序执行：

```
Copy → Compute
```

需要 Side Stream 和双缓冲才能与另一个 Batch 的计算重叠。

失败7：忽略NUMA

Pinned 了也不代表 Host Buffer 在离 GPU 最近的 Socket。

失败8：把Unified Memory理解成“没有传输”

页面仍可能迁移，甚至发生 Thrashing。

失败9：只看API耗时，不看GPU时间线

`non_blocking=True` 的 Python 调用很快返回，不代表 DMA 已完成。

失败10：为了CPU处理，把大Tensor频繁D2H

应先在 GPU 做 Reduction、Filtering、Top-k 或 Sampling，再返回小结果。

二十二、如何用 Profiler 验证

22.1 PyTorch Profiler

观察：

- `aten::to`；
- `aten::_to_copy`；
- `cudaMemcpyAsync`；
- `cudaStreamSynchronize`；
- DataLoader 等待；
- CPU Launch Gap。

22.2 Nsight Systems

重点看 Timeline：

```
CPU Thread
CUDA Copy Streams
CUDA Compute Streams
H2D / D2H Memcpy
Kernel
NCCL
CUDA API
```

验证问题：

1. H2D 是否在 GPU Compute 期间发生?
2. Copy 是否位于独立 Stream?
3. 是否存在隐式同步?
4. GPU Compute 前是否长期等待数据?
5. H2D 是否达到预期有效带宽?
6. D2H 是否让 CPU 主线程或默认 Stream 停顿?
7. 是否有大量小 Memcpy?

22.3 正确计时

CUDA 操作异步，不能只用普通 CPU 时钟包住调用：

```
start = time.perf_counter()
y = x.to("cuda", non_blocking=True)
elapsed = time.perf_counter() - start
```

这主要测到提交时间。

可使用 Event：

```
start = torch.cuda.Event(enable_timing=True)
end = torch.cuda.Event(enable_timing=True)

start.record()
y = x.to("cuda", non_blocking=True)
end.record()

end.synchronize()
print(start.elapsed_time(end), "ms")
```

端到端测量则在合理边界同步：

```
torch.cuda.synchronize()
t0 = time.perf_counter()

run_step()

torch.cuda.synchronize()
print(time.perf_counter() - t0)
```

二十三、工程选型决策表

场景	推荐路径	原因
常规训练输入	DataLoader Pinned + non_blocking H2D	简单、成熟、可流水化
GPU计算比H2D长	双缓冲	H2D可被完全隐藏
H2D比计算长	降低输入字节、提高链路、增加计算批量	Overlap后仍由H2D决定周期
很多小Tensor	合并/打包	摊薄API与DMA固定成本
CPU后处理只需少量结果	GPU先Reduction/Top-k	减少D2H Payload
Checkpoint保存	分片D2H + 后台IO；条件允许用GDS	避免主训练流长时间阻塞
参数/Optimizer Offload	预取 + Pinned Pool + Overlap	隐藏H2D/D2H，但占PCIe
不规则超大数据结构	Unified Memory + Prefetch评估	编程方便，但需防Page Fault/Thrash
稀疏一次性Host访问	Mapped Pinned/Zero-Copy评估	避免复制未使用的大部分数据
跨节点GPU通信	GPUDirect RDMA/NCCL	避免CPU Bounce Buffer
Storage直接喂GPU	GPUDirect Storage	降低CPU占用和Bounce Copy
双Socket多GPU	NUMA绑定	避免跨SocketHost内存流量

二十四、最终心智模型

```
CPU:
擅长控制、分支、IO、数据准备和低延迟小任务

GPU:
擅长大规模、规则、高并行、高算术强度计算

H2D:
把CPU内存数据送到GPU HBM

D2H:
把GPU结果返回CPU内存
```

```
Pageable:
通常需要内部Pinned Staging

Pinned:
允许Copy Engine直接DMA，是可靠异步传输的基础

non_blocking:
让CPU不立即等待，不等于自动产生GPU侧Overlap

真正Overlap:
Pinned + Async Copy + Side Stream + Event + 无依赖 + Double Buffer
```

固定问七个问题：

1. **必要性**：这份数据必须跨 CPU–GPU 边界吗?
2. **大小**：Payload 是多少 Byte？能否压缩、降精度或先 Reduction?
3. **粒度**：是否有大量小复制，应不应该合并?
4. **内存类型**：Pageable、Pinned、Managed 还是 GPU Memory?
5. **依赖**：谁什么时候第一次读取结果？需要什么 Event?
6. **重叠**：是否有独立计算可隐藏 Copy?
7. **拓扑**：PCIe/NVLink-C2C、NUMA、NIC、Storage 路径是什么?

真正的目标不是“让 memcpy 更快”，而是：

> **尽量删除跨边界传输，并将剩余传输从关键路径移到并行流水线中。**

可选自测

1. 计算题

输入：

```
Shape = [4,8192,4096]
Dtype = BF16
H2D有效带宽 = 28 GB/s
GPU计算 = 20 ms
```

计算：

- Batch 数据量的 MiB；

- H2D 时间；
- 串行每 Batch 时间；
- 完美双缓冲的稳态周期；
- 理想吞吐加速比。

2. 因果题

为什么：

```
x.pin_memory().to("cuda", non_blocking=True)
```

可能比：

```
x.to("cuda", non_blocking=True)
```

更慢？什么情况下 Pinning 才值得？

3. 工程决策题

LLM Decode 每 Step 产生：

```
Batch = 256
Vocab = 128K
FP16 Logits
```

CPU 只需要每个请求最终采样的 Token ID。

请比较：

1. 把完整 Logits D2H 后在 CPU Sampling；
2. 在 GPU 完成 Top-k/Softmax/Sampling，只 D2H Token ID。

计算两种方案的 Payload，并说明对 TPOT、PCIe、CPU 占用和 Batch 扩展性的影响。

继续学习

1. 前置：PCIe、NVLink、NVSwitch 与 NUMA 拓扑——理解链路、Root Complex 和跨Socket瓶颈。

2. 同层对比：CUDA Stream、Event、Copy Engine 与真正的 Overlap——深入异步依赖和时间线调试。
3. 下游应用：ZeRO/FSDP CPU Offload 与 NVMe Offload——理解参数、梯度和优化器状态如何在 CPU/GPU/存储间流水。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

事实来源与版本边界

截至 2026-08-03，本文稳定机制主要核对：

- NVIDIA CUDA C++ Programming Guide；
- NVIDIA CUDA C++ Best Practices Guide；
- PyTorch CUDA Semantics；
- PyTorch `pin_memory` / `non_blocking` 官方教程；
- NVIDIA GPUDirect RDMA / GPUDirect Storage 官方文档。

PCIe/NVLink 带宽、Copy Engine 数量、默认 Stream 行为、Unified Memory、一致性语义和 GPUDirect 支持依 GPU、CPU、驱动、操作系统、CUDA/PyTorch 版本与平台拓扑变化。生产环境必须以实际设备属性、`nvidia-smi topo -m`、CUDA Events 和 Nsight Systems Timeline 为准。